

CNT4713 Cryptography Project - Chat

Overview

Disclaimer: If you did not complete the chat and FTP projects you may wish to do so before starting this project. However, your grades for those projects will remain the same. All commands, flow, and operation from those projects are still needed as the base for connections and must remain unless specifically told to change in this project.

In this project you will create further extend your chat project to allow encrypted communication between client(s) and the server.

Constraints

- Use socket programming only.
- Use TCP sockets.
- Programming language: Python
- You **must** use the **cryptography** Python library
- No external messaging frameworks.
- Client and server must run as separate processes.
- Server file **must be named**: server.py
- Client file **must be named**: client.py
- You must explicitly follow the input and output guidelines.

An autograder will grade the code submissions. This autograder executes pre-written versions of the client and server separately with your submission (your client + known server, your server + known client). You must therefore explicitly name the files as instructed. Also, you must explicitly follow the commands, flow, input, and output given. **From a runtime perspective, the usernames, ports, IP addresses, messages, and file names are all variable.** The grader will use their own values for these settings and the expected out **must** match. All submissions which do not match will received a **zero**.

The Server

The following table outlines the commands the server will accept. All commands will return status code **200** when the server executes the command successfully, and **500** when the command fails.

Command	Example
connect <ip> <port>	connect 192.168.254.253 8991
login <username>	login alice
who	who
broadcast <message>	broadcast Hello all!
private <username> <message>	private bob Meet after class.

quit	quit
------	------

Responses from the server **must all have the following format:**

Status code

<EMPTY LINE>

Data section if required

Further explanation on the commands is found below:

1. The server will be started via command line with a user provided TCP port. This will be called the CONTROL PORT. For example:

- i. `python server.py 8991`

2. On start, the server will create an **RSA 2048 public-private key pair** and store it in memory.

3. The server will listen for the following **commands** (in bold) from the client and:

- a. **connect** – The connect command changes slightly. The server will establish the connection with the client and respond with a different port number, the DATA PORT, for the client to receive data **and also sends its own public key**. The server creates a new socket for this data connection.

Expected behavior:

- i. All commands will be received on the CONTROL PORT.
 - ii. The response to the connect command will be sent to the client port provided in the connect connection handshake.
 - iii. Responses to other commands will be sent on the DATA PORT.

Example: the server may respond stating successful connection and to use data port 9000:

200

9000

<public key>

- b. **login** – The server registers a user with the server and keeps track of the user appropriately to send future messages to.

Expected behavior:

- i. Server **decrypts** the message.
 - ii. Server validates uniqueness of username and stores the **client's public key**. Sends failure and stops processing request if name not unique or any components of the expected message are missing.
 - iii. User is added to active client list.

- iv. All further commands received and responses sent must be encrypted appropriately
- v. Server broadcasts user join notification.

Example - the server receives this message:

```
login
```

```
alice
<public key>
```

The server broadcasts:

```
200
```

```
join
alice
```

- c. **All** future messages between each client and the server must be sent encrypted and be decrypted using the appropriate client specific keys to determine how to process and route the message. The server computes the SHA256 hash of the message, encrypts it using the correct key, and responds to the client with the encrypted hash (the cipher text). The server responds on the data socket.

The Client

1. The client will be started via command line. For example:

- i. `python client.py`

On start, the client will create a public-private key pair and store it in memory.

2. The user will then type the command to connect to the server using its IP address and the CONTROL PORT. For example:

- i. `connect 192.168.254.253 8991`

3. On server response, the client will:

- a. Extract the data port and create a new socket for data connections
 - b. Extract the servers public key from the response and store it in memory for future use

4. The client proceeds with the **login** command. It sends the username it wishes to be known as to the server and its public key all encrypted with the servers public key.

Example message format:

```
login
```

```
alice
<public key>
```

5. The client will receive a broadcast message stating it successfully joined.
6. The client(s) and the server, will now send and process encrypted messages using their own asymmetric key pairs:
 - a. The client will create a message, encrypt it using the server's public key, and send it to the server using the **broadcast** command.
 - b. The server will decrypt the message and then broadcast it to all connected clients using their own public keys.
 - c. The clients will decrypt the messages and display them to the screen.

Output

Explicitly use the following output guideline template. “>” at the start of a line indicates a stage requiring user input or intervention:

1. The Server

```
> python server.py 8991
Starting server...
Creating RSA keypair
RSA keypair created
Creating server socket
Awaiting connections...
Connection requested. Creating data socket
Received encrypted message
Login requested by: bob
Connection requested. Creating data socket
Received encrypted message
Login requested by: alice
Received encrypted message
Who requested. Sending users.
Received encrypted message
Broadcast requested by alice
Message: Hello all!
Received encrypted message
Private message from alice to bob
Received encrypted message
Quit requested by alice
Received encrypted message
Quit requested by bob
```

2. The Sending Client

```
> python client.py
```

```
Starting client...
```

```
> connect 192.168.254.253 8991
```

```
200 status code received. Starting data connection on port  
<DATA PORT>
```

```
> login alice
```

```
Received encrypted message
```

```
200 status code received. Login successful
```

```
> who
```

```
Received encrypted message
```

```
200 status code received. Users currently connected: bob,  
mallory
```

```
> broadcast Hello all!
```

```
Received encrypted message
```

```
200 status code received.
```

```
Broadcast message from alice: Hello
```

```
> private bob Let's talk
```

```
Received encrypted message
```

```
200 status code received. Message sent.
```

```
> quit
```

```
Received encrypted message
```

```
200 status code received.
```

3. The Receiving Client

```
> python client.py
Starting client...
> connect 192.168.254.253 8991
200 status code received. Starting data connection on port
<DATA PORT>
> login bob
Received encrypted message
200 status code received. Login successful
Received encrypted message
200 status code received.
Broadcast message from alice: Hello
Received encrypted message
200 status code received.
alice: Let's talk
> quit
Received encrypted message
200 status code received.
```

Submission

1. In a text file names **answers.txt**, answer the following question(s):
 - a. What is one major difference between the protocol in this project when compared to the TLS protocol?
 - b. What flaws can you identify in the “secure” protocol?
 - c. How could the “secure” server and client framework above be compromised?
 - d. How could you improve the framework? Be specific with your answers.

Do note: No points will be awarded for “Just use X protocol or framework”.
2. A copy of your final code to the LMS. (**Source code only (.py files). No zips. No IDE project files**). Do **not** submit zip files to file share site or links to zips at file share sites (including Google Drive). **Zero will be awarded without chance for review.**

3. A video recording (screen recording) of the execution of your team's project. The recording must show your application being executed for each graded part of the project. Narrate the video stating what is being shown.
4. Use Wireshark to show the messages being sent between the client(s) and the server. Ensure to show and highlight the message formats stated in the requirements. Wireshark will be a portion of the video grade.

You may upload the video to YouTube or Google Drive and submit the link to the LMS. **The link must be viewable at time of grading. Zero with no review will be awarded if it is not viewable at time of grading.**

Grading

The table below shows how the project will be graded. **Grading shown in the LMS and instructions from your instructor override all grading shown below.**

Item	Percentage
Server connect	6
Server login command	6
Server who command	6
Server broadcast	6
Server private	6
Server quit	5
Client connect command	6
Client login command	6
Client who command	6
Client broadcast	6
Client private	6
Client quit	5
Answers to questions	5
video	25

Code plagiarism will be reported for academic dishonesty.